A0 Start here: foundations for Databricks

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2

A0 Start here: foundations for Databricks 3

A0-L1 Spark DataFrames, the model and the core methods 3

A0-L2 The SQL that pipelines use 7

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers 10

Track A best-practice checklist 15

Appendix: solutions and answer keys 16

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A0 Start here: foundations for Databricks

Roadmap: Prerequisite for all months. Spark DataFrames, the SQL pipelines use, and the lakehouse map: Databricks, Unity Catalog, Delta and medallion layers.

A0-L1 Spark DataFrames, the model and the core methods

Concept

Databricks runs **Apache Spark**, which splits work across many machines. You work with **DataFrames**: tables with named, typed columns, spread over the cluster. Examples use PySpark 4.0, as in Databricks Runtime 17.3 LTS. A notebook gives you a ready `spark` session.

```
1 from pyspark.sql import functions as F
2
3 df = spark.table("research_prod.research_silver.nat_respondents")
```

`spark.createDataFrame(rows, "region STRING, w DOUBLE")` builds a small DataFrame from Python tuples, which is handy in tests.

Transformations describe a new DataFrame and run nothing yet:

- `select("region", "w")` keeps columns.
- `filter(F.col("w") > 0)` keeps rows. A boolean column filters directly: `filter(F.col("in_scope"))`.
- `withColumn("w2", F.col("w") * 2)` adds or replaces a column. `F.lit(1)` is a constant, and `F.when(cond, a).otherwise(b)` picks a value per row.
- `groupBy("region").agg(F.sum("w").alias("total"), F.count("*").alias("n"))` summarises per group.
- `join(other, "region_code", "left")` and `orderBy("region")` join and sort.

Actions run the plan and return a result: `show()` prints rows, `count()` counts them, `collect()` returns a list of `Row` objects to Python, and `write` saves a table. Spark is **lazy**: nothing happens until an action, so a typo in a column name may only fail at the action.

DataFrames are **immutable**. Every transformation returns a new DataFrame and leaves the old one unchanged, so always use the result: `df = df.withColumn(...)` or `return df.withColumn(...)`.

Import the functions module as `F`. Importing `sum` or `max` from it by name hides Python's own `sum` and `max`. Collect only small results: `collect()` pulls every row onto one machine.

Why it matters

A junior engineer wrote `df.withColumn("wave", F.lit("W14"))` on its own line and saved `df`. The wave column never existed, because the result was thrown away. Every later job that grouped by wave failed, and a morning was spent looking at the wrong notebook. Assigning the result, `df = df.withColumn(...)`, is the whole fix.

Worked example: weak code

```
1 from pyspark.sql.functions import *
2
3 df = spark.table("research_prod.research_silver.nat_respondents")
2 df.filter(col("in_scope") == True)
3 rows = df.collect()
6 total = sum(r.final_weight for r in rows)
```

1. **Line 1.** Imports Spark's sum, max and others over Python's built-ins.
2. **Line 4.** The filtered DataFrame is thrown away, and == True is redundant.
3. **Line 5.** Pulls every row onto the driver, then adds them up in Python.

Worked example: strong code

```
1  from pyspark.sql import DataFrame
1  from pyspark.sql import functions as F
3
4
5  def weighted_base(df: DataFrame) -> DataFrame:
6      """Weighted base and respondent count per region, in-scope rows
       only."""
7      return (
2          df.filter(F.col("in_scope"))
9          .groupBy("region")
3          .agg(F.sum("final_weight").alias("weighted_base"),
       F.count("*").alias("n"))
11     )
```

1. **Line 2.** Spark functions stay behind F., so Python's built-ins keep their meaning.
2. **Line 8.** A boolean column filters directly, and the result is used.
3. **Line 10.** The cluster does the summing. Only the small result ever needs collecting.

Chain transformations and use their results, let Spark aggregate, and run an action only when you need the answer.

Recap card

- **Transformation against action** Transformations (select, filter, withColumn, groupBy, join) describe a new DataFrame. Actions (show, count, collect, write) run the plan.
- **Why df.withColumn(...) on its own line does nothing** DataFrames are immutable. Use the result: df = df.withColumn(...).
- **Summarise per group** df.groupBy("region").agg(F.sum("w").alias("total"), F.count("*").alias("n")).
- **How to import Spark functions** from pyspark.sql import functions as F. Never import sum, max or min from it by name.

Practice

Exercise 1. Multiple choice (level 1)

Which line makes Spark read data and compute?

- a) `kept = df.filter(F.col("w") > 0)`
- b) `totals = kept.groupBy("region").agg(F.sum("w").alias("total"))`
- c) `totals.count()`
- d) `totals = totals.orderBy("region")`

Why?

- r1) count() is an action. The other lines are transformations that only describe the plan.
- r2) count() is the last line.

Exercise 2. Predict the output (level 2)

What does this print? (PySpark 4.0)

```
from pyspark.sql import functions as F

df = spark.createDataFrame([("N", 1.0), ("N", 2.0), ("S", 4.0)], "region STRING, w DOUBLE")
out = df.filter(F.col("w") > 1).groupBy("region").agg(F.sum("w").alias("total")).orderBy("region")
print([(r.region, r.total) for r in out.collect()])
```

Exercise 3. Fill the gap (level 2)

Total the positive weights per region.

```
totals = df.filter(F.col("w") > 0).groupBy("region")._____(1)(F._____(2)("w").alias("total"))
```

Exercise 4. Write the function (level 3)

Write `region_totals(df: DataFrame) -> DataFrame`. Keep rows that are in scope (the boolean column `in_scope`) and have a positive `w`. Return one row per `region` with `n`, the number of rows, and `total_weight`, the sum of `w`. Use type hints and a docstring. Do not collect.

Spark does not run in the browser: the checker reads your code, and the CI runs the reference solution on real Spark.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Row count and weight total per region, for in-scope rows with a positive weight.
def region_totals(df: DataFrame) -> DataFrame: ...
```

Starter code:

```
def region_totals(df):
    rows = df.collect()
    out = {}
    for r in rows:
        if r.in_scope == True and r.w > 0:
            out[r.region] = out.get(r.region, 0) + r.w
    return out
```

Exercise 5. Spot the bug (level 3)

The returned DataFrame never has a `weight_band` column. Tap the line with the bug, then fix the function.

```
def add_weight_band(df: DataFrame) -> DataFrame:
    """Add weight_band: high above 3, otherwise normal."""
    df.withColumn("weight_band", F.when(F.col("w") > 3, "high").otherwise("normal"))
    return df
```

Write down the line number of the bug. Then write the fix.

Sources

- Transformations return DataFrames without running; count and collect run the plan; withColumn leaves the original DataFrame unchanged; the E2 code prints [('N', 2.0), ('S', 4.0)]. (verified: Run on PySpark 4.0.4 locally.)

A0-L2 The SQL that pipelines use

Concept

Databricks pipelines mix PySpark with Spark SQL. The examples use Spark 4.0 SQL. Run SQL from Python with `spark.sql("SELECT ...")`, which returns a DataFrame.

Read and filter. `SELECT region, final_weight FROM respondents WHERE final_weight > 0` picks columns and keeps rows.

Summarise. `GROUP BY region` makes one row per region. Aggregates work per group: `COUNT(*)` counts rows, `COUNT(final_weight)` counts non-NULL values, and `SUM` and `AVG` skip NULLs. `HAVING SUM(final_weight) > 100` filters groups after they are summarised, where `WHERE` filters rows before.

Join. `JOIN regions g ON r.region_code = g.region_code` keeps only matching rows. `LEFT JOIN` keeps every row of the left table and fills the right side with NULL when there is no match. `LEFT ANTI JOIN` keeps the left rows that have **no** match, which finds orphan keys in one line.

NULL means unknown. Any comparison with NULL is unknown, never true, so `WHERE final_weight = NULL` matches nothing. Use `IS NULL` and `IS NOT NULL`.

Pick one row per key. A delivery can repeat a respondent. A **window function** numbers the rows within each key, newest first, and you keep number 1:

```
1 SELECT respondent_id, wave_id, final_weight
2 FROM (
3   SELECT *,
4          ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
    delivery_version DESC) AS rn
5   FROM bronze_deliveries
6 )
7 WHERE rn = 1
```

`PARTITION BY` sets the groups, `ORDER BY ... DESC` puts the newest first, and `ROW_NUMBER()` gives exactly one row the number 1 in each group. `DISTINCT` would keep both versions, because their values differ.

Lessons A3 and A4 add `MERGE`, `GRANT` and row filters on top of these.

Why it matters

A data-quality check counted missing weights with `WHERE final_weight = NULL`. It reported zero every night for two months while 1,800 weights were

missing, because a comparison with NULL is never true. `IS NULL` found them on the first run, and the check now catches the problem it was written for.

Worked example: weak code

```
1 SELECT DISTINCT *
2 FROM bronze_deliveries
3 WHERE final_weight != NULL;
```

1. **Line 1.** Two versions of one respondent differ in their values, so DISTINCT keeps both.
2. **Line 3.** A comparison with NULL is never true, so this returns no rows at all.

Worked example: strong code

```
1 SELECT respondent_id, wave_id, final_weight
2 FROM (
3   SELECT *,
4          ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
    delivery_version DESC) AS rn
5   FROM bronze_deliveries
6   WHERE final_weight IS NOT NULL
7 )
8 WHERE rn = 1;
```

1. **Line 4.** Numbers each respondent's versions, newest first.
2. **Line 6.** IS NOT NULL is the test that works for NULL.
3. **Line 8.** Exactly one row per respondent and wave: the latest version.

Filter rows with WHERE and groups with HAVING, test NULL with IS NULL, and keep one row per key with ROW_NUMBER, not DISTINCT.

Recap card

- **WHERE against HAVING** WHERE filters rows before grouping. HAVING filters groups after aggregation.
- COUNT() against COUNT(column) COUNT() counts rows. COUNT(column) counts non-NULL values. SUM and AVG skip NULLs.
- **Find rows with no match in a lookup** LEFT ANTI JOIN lookup ON key. A LEFT JOIN keeps all rows and fills NULLs.
- **Test for a missing value in SQL** IS NULL or IS NOT NULL. = NULL is never true.
- **Keep the latest row per key** ROW_NUMBER() OVER (PARTITION BY keys ORDER BY version DESC) AS rn, then WHERE rn = 1.

Practice

Exercise 1. Multiple choice (level 1)

Which condition finds respondents whose weight is missing?

- a) `WHERE final_weight = NULL`
- b) `WHERE final_weight IS NULL`
- c) `WHERE final_weight = 0`
- d) `WHERE NOT final_weight`

Why?

- r1) A comparison with NULL is unknown, never true. IS NULL is the test made for it.
- r2) IS is faster than =.

Exercise 2. Predict the output (level 2)

`respondents` has three rows, with final_weight 1.0, NULL and 2.0. What does this return? Answer as two numbers.

```
SELECT COUNT(*), COUNT(final_weight)
FROM respondents;
```

Exercise 3. Fill the gap (level 2)

List respondents whose region code is missing from the regions lookup.

```
SELECT r.respondent_id, r.region_code
FROM respondents r
LEFT _____(1) JOIN regions g
  _____(2) r.region_code = g.region_code;
```

Exercise 4. Write the function (level 3)

`bronze_deliveries` can hold several versions of a respondent in a wave. Write a query that returns `respondent_id`, `wave_id`, `delivery_version` and `final_weight` for the latest version only: the highest `delivery_version` per respondent and wave. Use `ROW_NUMBER()`.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
SELECT ...
...
  SELECT ...
         ...
  FROM ...
...
WHERE ...
```

Starter code:

```
SELECT DISTINCT respondent_id, wave_id, delivery_version, final_weight
FROM bronze_deliveries;
```

Exercise 5. Spot the bug (level 3)

This check reports 0 missing weights every night, though weights are missing. Tap the line with the bug, then fix it.

```
SELECT COUNT(*) AS missing_weights
FROM research_prod.research_silver.nat_respondents
WHERE final_weight = NULL;
```

Write down the line number of the bug. Then write the fix.

Sources

- COUNT(*) is 3 and COUNT(col) is 2 with one NULL; WHERE col = NULL matches no rows; LEFT ANTI JOIN returns unmatched left rows; the ROW_NUMBER query keeps the highest delivery_version per respondent and wave. (verified: Run on Spark 4.0.4 SQL locally.)

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers

Concept

Track A uses a handful of Databricks words on every page. This lesson places each one.

Workspace, notebooks, compute, jobs. You work in a **workspace**, writing code in **notebooks**. Code runs on **compute**: **serverless** compute that Databricks manages for you, or a cluster you configure. Databricks Free Edition offers serverless compute only. A **job** runs notebooks on a schedule, with parameters (lesson A10-L2).

Unity Catalog organises data and permissions with three-level names, `catalog.schema.table`:

- a **catalog** per environment, such as `research_dev` and `research_prod`,
- a **schema** per layer, such as `research_silver`,
- a **table**, such as `nat_respondents`.

Volumes hold files, such as raw delivery CSVs, under the same names and permissions. Grants go to groups, never to individual users (lesson A4-L2).

Delta tables. A table created `USING DELTA`, the Databricks default, is a folder of Parquet data files plus a transaction log, `_delta_log`. Each write adds one numbered JSON commit to the log, so the table has versions 0, 1, 2 and so on. `DESCRIBE HISTORY t` lists them, and `SELECT * FROM t VERSION AS OF 1` reads an older one (lesson A3-L3). Readers never see a half-finished write.

Medallion layers give each table one job:

- **Bronze** keeps every delivery as received, plus audit columns: `nat_raw`.
- **Silver** has clean names and types and one row per respondent per wave: `nat_respondents`.
- **Gold** holds business-ready aggregates with a stated grain: `nat_metrics`.

Data moves forward only. A fix in silver logic means rebuilding silver from bronze, never editing bronze. Module A2 sets the contract for each layer.

Why it matters

Two analysts both wrote to a table called `respondents`, one in the development catalog and one in production, because their notebooks used two-part names and different default catalogs. Production silver received a week of test rows before anyone noticed. Three-level names, with the catalog stated explicitly or set from a job parameter, make the target impossible to mistake.

Worked example: weak code

```
1 CREATE TABLE respondents AS SELECT * FROM csv.`/Users/anna/w14.csv`;
2 UPDATE respondents SET final_weight = 1.0 WHERE final_weight IS NULL;
```

1. **Line 1.** A one-part name lands in whatever catalog and schema are current, read from a personal folder.
2. **Line 2.** Edits the only copy of the delivery in place, so the original values are lost.

Worked example: strong code

```
1 -- Bronze: the delivery as received, from a governed volume
1 CREATE TABLE IF NOT EXISTS research_prod.research_bronze.nat_raw
3 AS SELECT *, current_timestamp() AS ingestion_timestamp
2 FROM csv.`/Volumes/research_prod/landing/deliveries/w14.csv`;
5
6 -- Every write is a version you can inspect
3 DESCRIBE HISTORY research_prod.research_bronze.nat_raw;
```

1. **Line 2.** Catalog, schema and table spelled out: production, bronze layer, the tracker's raw table.

2. **Line 4.** Files live in a Unity Catalog volume, with the same permissions as tables.
3. **Line 7.** The Delta log shows every version, who wrote it and what changed.

Name every table in full, keep files in volumes, let Delta keep versions, and move data forward from bronze to silver to gold.

Recap card

- **Unity Catalog name** catalog.schema.table, such as research_prod.research_silver.nat_respondents. Files live in volumes.
- **What makes a Delta table** Parquet data files plus _delta_log, where each write adds a numbered commit: versions 0, 1, 2.
- **Read an older version of a table** SELECT * FROM t VERSION AS OF 1. DESCRIBE HISTORY t lists the versions.
- **The three medallion layers** Bronze: as received, with audit columns. Silver: clean, one row per key. Gold: aggregates with a stated grain.

Practice

Exercise 1. Multiple choice (level 1)

In `research_prod.research_silver.nat_respondents`, what is `research_silver`?

- a) The catalog
- b) The schema
- c) The table
- d) The workspace

Why?

- r1) Unity Catalog names run catalog.schema.table, so the middle part is the schema.
- r2) Names containing silver are always schemas.

Exercise 2. Predict the output (level 2)

These statements run in order on a new Delta table. What does the last query return?

```
1 CREATE TABLE t (id INT) USING DELTA;   -- version 0
2 INSERT INTO t VALUES (1);              -- version 1
3 INSERT INTO t VALUES (2);              -- version 2
4 SELECT id FROM t VERSION AS OF 1;
```

Exercise 3. Fill the gap (level 2)

Name the NAT tracker's tables by the team convention.

```
bronze: research_prod.research_bronze.nat_____(1)
silver: research_prod.research_silver.nat_____(2)
gold:   research_prod.research_gold.nat_____(3)
```

Exercise 4. Write the function (level 3)

Write `parse_table_name(name: str) -> tuple[str, str, str]` for a job that must only ever write to fully named tables. Split the name on dots and return `(catalog, schema, table)`. Raise `ValueError` naming the input when there are not exactly three parts, or when any part is empty.

Your code must pass these checks:

- Return the catalog, schema and table as a tuple of three strings.
- Raise ValueError naming the input when the name has fewer than three parts.
- Raise ValueError for more than three parts, or for an empty part.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Split a Unity Catalog table name into catalog, schema and table.
def parse_table_name(name: str) -> tuple[str, str, str]: ...
```

Starter code:

```python
def parse_table_name(name):
    return tuple(name.split("."))
```

Exercise 5. Architecture design (level 3)

Place each item for the NAT tracker.

Options: A Unity Catalog volume, Bronze table nat_raw, Silver table nat_respondents, Gold table nat_metrics.

1. The supplier's delivery file, w14.csv, exactly as sent
2. Every row of every delivery, with ingestion_id and source_file
3. One typed row per respondent per wave, latest delivery only
4. Weighted reach by wave and region

Sources

- A Delta table's _delta_log holds one JSON commit per write (00000000000000000000.json, ...); DESCRIBE HISTORY lists versions 0, 1, 2 after a create and two inserts; VERSION AS OF 1 returns the first inserted row. (verified: Run on Spark 4.0.4 with delta-spark 4.0.1 locally.)

- Databricks Free Edition offers serverless compute only. (verified: docs.databricks.com/aws/en/getting-started/free-edition-limitations (checked 2026-09-25, VERSIONS.md).)

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A0-L1 Spark DataFrames, the model and the core methods

A0-L1 exercise 1

Hints. (1) Spark is lazy. (2) Which call returns a result to Python instead of a new DataFrame? (3) count returns a number.

Answer. c) `totals.count()` Because: r1) count() is an action. The other lines are transformations that only describe the plan.

Transformations build a plan. An action such as `count()`, `show()`, `collect()` or a write runs it.

Common wrong answers.

- `a`: filter is a transformation. It returns a new DataFrame and runs nothing yet.
- `b`: groupBy and agg are transformations too. Spark waits for an action.
- `d`: orderBy is a transformation. The plan grows, nothing runs.
- `c|r2`: Right line, wrong reason. Position does not matter. count() is an action.

A0-L1 exercise 2

Hints. (1) Apply the filter first. (2) Which rows have w above 1? (3) Then add up w per region.

Answer.

```
1 [('N', 2.0), ('S', 4.0)]
```

The filter keeps 2.0 for N and 4.0 for S, the groupBy sums them, and orderBy puts N first.

Common wrong answers.

- `[('N', 3.0), ('S', 4.0)]`: The filter runs first: w > 1 drops the row with 1.0, so N totals 2.0.
- `[('S', 4.0), ('N', 2.0)]`: orderBy("region") sorts N before S.

A0-L1 exercise 3

Hints. (1) The method that follows groupBy. (2) The function that adds values. (3) `agg` and `sum`.

Answer.

```
totals = df.filter(F.col("w") > 0).groupBy("region").agg(F.sum("w").alias("total"))
```

`groupBy(...).agg(...)` summarises each group. `F.sum("w")` adds the weights, and `alias` names the result.

Common wrong answers.

- g1 = `agg`; g2 = `count`: F.count counts rows. The task asks for the total weight: F.sum.
- g1 = `select`; g2 = `sum`: After groupBy, summarise with agg.

A0-L1 exercise 4

Hints. (1) Filter with `F.col("in_scope") & (F.col("w") > 0)`, brackets around the comparison. (2) Then `.groupBy("region")`. (3) `.agg(F.count("*").alias("n"), F.sum("w").alias("total_weight"))`.

Answer.

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def region_totals(df: DataFrame) -> DataFrame:
    """Row count and weight total per region, for in-scope rows with a positive weight.

    Args:
        df: Respondents with region, in_scope and w columns.

    Returns:
        One row per region with n and total_weight.
    """
    return (
        df.filter(F.col("in_scope") & (F.col("w") > 0))
        .groupBy("region")
        .agg(F.count("*").alias("n"), F.sum("w").alias("total_weight"))
    )
```

One chain of transformations: filter, group, aggregate. The cluster does the work, and the function returns a DataFrame for the caller to write or show.

A0-L1 exercise 5

Hints. (1) What does withColumn return? (2) DataFrames never change in place. (3) `return df.withColumn(...)`.

Answer. The bug is on line 3.

Fixed version:

```
1 def add_weight_band(df: DataFrame) -> DataFrame:
2     """Add weight_band: high above 3, otherwise normal."""
3     return df.withColumn("weight_band", F.when(F.col("w") > 3,
    "high").otherwise("normal"))
```

`withColumn` returns a new DataFrame. Line 3 discards it, so the function returns the original. Return the new DataFrame, or assign it back to `df`.

Common wrong answers.

- Line 4: Returning df is right once df holds the new DataFrame. Which line throws the new DataFrame away?

A0-L2 The SQL that pipelines use

A0-L2 exercise 1

Hints. (1) What does comparing anything with NULL give? (2) Unknown, never true. (3) SQL has a dedicated test.

Answer. b) `WHERE final_weight IS NULL` Because: r1) A comparison with NULL is unknown, never true. IS NULL is the test made for it.

NULL means unknown, so `= NULL` is never true. `IS NULL` matches missing values.

Common wrong answers.

- `a`: = NULL is never true, so it matches no rows, even missing ones.
- `c`: 0 is a real weight, not a missing one.
- `d`: NOT on a NULL is still unknown, so missing rows are not kept.
- `b|r2`: Right condition, wrong reason. It is about correctness: = NULL never matches.

A0-L2 exercise 2

Hints. (1) What does COUNT(*) count? (2) What does COUNT(column) skip? (3) Rows against non-NULL values.

Answer.

```
1 3 2
```

`COUNT(*)` counts all three rows. `COUNT(final_weight)` counts the two non-NULL weights.

Common wrong answers.

- `3 3`: COUNT(column) counts only non-NULL values, so it skips the missing weight.
- `2 2`: COUNT(*) counts rows, whatever their values.

A0-L2 exercise 3

Hints. (1) A join that keeps left rows with no match. (2) Joins name their condition after one keyword. (3) `ANTI` and `ON`.

Answer.

```sql
SELECT r.respondent_id, r.region_code
FROM respondents r
LEFT ANTI JOIN regions g
  ON r.region_code = g.region_code;
```

`LEFT ANTI JOIN ... ON` returns the left rows with no match on the right: the orphan codes.

Common wrong answers.

- g1 = `OUTER`; g2 = `ON`: A LEFT OUTER JOIN keeps every respondent. An anti join keeps only those with no match.
- g1 = `SEMI`; g2 = `ON`: A semi join keeps rows that do match. You want the ones that do not.

A0-L2 exercise 4

Hints. (1) Put the numbering in a subquery: `SELECT *, ROW_NUMBER() OVER (...) AS rn FROM bronze_deliveries`. (2) Inside OVER: `PARTITION BY respondent_id, wave_id ORDER BY delivery_version DESC`. (3) Outside, select the four columns and keep `WHERE rn = 1`.

Answer.

```sql
SELECT respondent_id, wave_id, delivery_version, final_weight
FROM (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
    delivery_version DESC) AS rn
  FROM bronze_deliveries
)
WHERE rn = 1;
```

`ROW_NUMBER()` numbers each respondent's versions within a wave, newest first, and `WHERE rn = 1` keeps one row per key. Without `DESC` you keep the

oldest version, and without `wave_id` you keep one row per respondent across all waves.

A0-L2 exercise 5

Hints. (1) Which rows does the WHERE clause keep? (2) Is any comparison with NULL ever true? (3) `WHERE final_weight IS NULL`.

Answer. The bug is on line 3.

Fixed version:

```
1 SELECT COUNT(*) AS missing_weights
2 FROM research_prod.research_silver.nat_respondents
3 WHERE final_weight IS NULL;
```

`= NULL` is unknown for every row, so no row passes the filter and the count is always 0. `IS NULL` counts the missing weights.

Common wrong answers.

- Line 1: COUNT(*) counts the rows the WHERE clause keeps. The WHERE clause keeps none.
- Line 2: The table name is a valid three-level name.

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers

A0-L3 exercise 1

Hints. (1) Three parts, three levels. (2) catalog.schema.table. (3) The middle one.

Answer. b) The schema Because: r1) Unity Catalog names run catalog.schema.table, so the middle part is the schema.

Unity Catalog names are `catalog.schema.table`. Here the catalog is the environment, the schema is the layer and the table is the tracker's silver table.

Common wrong answers.

- `a`: The catalog comes first: research_prod, the production environment.
- `c`: The table comes last: nat_respondents.
- `d`: A workspace is where you work. It is not part of a table name.
- `b|r2`: Right part, wrong reason. Position decides: catalog.schema.table.

A0-L3 exercise 2

Hints. (1) Each statement that changes the table makes one version. (2) What did the table hold at version 1? (3) Only the first insert had run.

Answer.

```
1
```

Version 1 is the table right after the first insert, so it holds the single row with id 1.

Common wrong answers.

- `1 2`: Version 1 is the table after the first insert. The second insert made version 2.
- `2`: VERSION AS OF 1 reads the table as it was after the first insert, which held only 1.

A0-L3 exercise 3

Hints. (1) Bronze holds the deliveries as received. (2) Silver holds one row per respondent per wave. (3) `raw`, `respondents`, `metrics`.

Answer.

```
bronze: research_prod.research_bronze.nat_raw
silver: research_prod.research_silver.nat_respondents
gold:   research_prod.research_gold.nat_metrics
```

The convention is `[tracker]_raw`, `[tracker]_respondents` and `[tracker]_metrics`, so a table's name says what it holds and the schema says its layer.

Common wrong answers.

- g1 = `bronze`; g2 = `silver`; g3 = `gold`: The layer is already in the schema name. The table name says what the table holds: raw, respondents, metrics.

A0-L3 exercise 4

Hints. (1) `parts = name.split(".")`. (2) Check `len(parts) != 3` and whether any part is an empty string. (3) Put the name in the message: `f"expected catalog.schema.table, got {name!r}"`.

Answer.

```
def parse_table_name(name: str) -> tuple[str, str, str]:
    """Split a Unity Catalog table name into catalog, schema and table.

    Args:
```

```
        name: A name such as
    research_prod.research_silver.nat_respondents.

    Returns:
        (catalog, schema, table).

    Raises:
        ValueError: If the name does not have exactly three non-empty
    parts.
    """
    parts = name.split(".")
    if len(parts) != 3 or any(part == "" for part in parts):
        raise ValueError(f"expected catalog.schema.table, got {name!r}")
    catalog, schema, table = parts
    return catalog, schema, table
```

A job that refuses anything but a full, three-part name can never write to whatever catalog happens to be the default.

A0-L3 exercise 5

Hints. (1) A file is not a table. (2) As received, then clean, then aggregated. (3) Volume, bronze, silver, gold.

Answer.

- Files as sent: a volume.
- Rows as received, with audit columns: bronze.
- Clean, deduplicated respondents: silver.
- Business-ready aggregates: gold.

Files sit in a volume, and each table layer has one job: bronze as received, silver clean, gold aggregated.

Common wrong answers.

- raw_rows = `silver`: Silver keeps one row per respondent per wave. Every delivered row, duplicates included, belongs in bronze.
- clean = `gold`: Gold holds aggregates. Clean respondent-level rows are silver.